Payload Decoding Guide

MeteoWind IoT Pro Gen2 | How to parse the payloads
Firmware 1.02.011

1. What this document is for

This is a decoder-writer's guide for the MeteoWind IoT Pro Gen2. It covers every LoRaWAN payload the station sends, in the order you need them to write a parser: how the bits are laid out, how each field converts to a physical value, and the four encodings that produce wrong readings if you decode them naively. Section 7 adds the alarm-settings downlink, which is bit-packed the same way, because building it is the same skill in reverse. A complete, tested JavaScript decoder is included in section 8.
The document is self-contained: everything needed to decode the station's uplinks and to build an alarm-settings command is here. The field layouts were taken from the firmware itself and the decoder in section 8 was verified against it, so where this guide disagrees with an older payload sheet, this guide describes what firmware 1.02.011 actually does.

1.1 What this document covers

Port	Message	Size	Direction	Section
1	Periodic measurement	14 B	uplink	4
3	Alarm event	6 B	uplink	5
2	Service message MW1S - device identity	12 B	uplink	6.1
2	Service message MW2S - alarm settings readback	12 B	uplink	6.2
2	Service message MW0S - position, not implemented	12 B	uplink	6.3
4	Retransmission of a stored payload	14 or 6 B	uplink	4 / 5
10	Alarm-settings command	3 B	downlink	7

A payload on port 4 is a byte-for-byte replay of one the station sent earlier on port 1 or port 3, resent on request. It carries no extra header, so decode it by length: 14 bytes uses the periodic layout of section 4, 6 bytes the alarm layout of section 5. The index field inside identifies which original message it is.
Port 50 carries short replies to configuration commands. They are bare hex fields with no message identifier, so they can only be interpreted against the command that was just sent, and they are outside the scope of this guide.

2. The parsing model

Every payload is one continuous bitstream. Fields are packed most-significant-bit first, back to back, with no padding or alignment between them, so most fields straddle byte boundaries. Only the first field of the periodic payload happens to start and end on a byte.

Do not try to parse these payloads by slicing bytes. A byte-oriented parser works for the index field and then silently produces nonsense for everything after it. Read bit ranges.

2.1 Four steps

Step	What to do
1	Take the raw bytes and treat them as one bitstream, bit 1 being the most significant bit of byte 0
2	For each field in table order, read its width in bits, most significant bit first, into an unsigned integer - call it `raw`
3	If every bit of the field is set, the reading is INVALID. Emit null and stop processing that field. See 3.1
4	Otherwise the field value is `raw * resolution + minimum`, using the resolution and minimum from the field table

Step 4 gives the transmitted quantity, which for the wind speeds is a click frequency in Hz and not a speed. Two further steps turn those into wind speeds: rebuild the gusts from the additions (3.3), then apply the anemometer calibration (3.4).

2.2 A payload decoded by hand

Periodic payload `039348F8709030ED7A08B1A4294C` on port 1. Writing the 14 bytes out as 112 bits and cutting them at the field widths from section 4 gives:

Bits	Field	Bits read	raw	Value	Unit
1-8	Index	`00000011`	3	3	
9	Battery in slot	`1`	1	1	
10-21	Wind speed average	`001001101001`	617	12.34	Hz
22-30	3s gust addition	`000111110`	62	6.2	Hz
31-38	1s gust addition	`00011100`	28	2.8	Hz
39-47	3s minimum	`001001000`	72	7.2	Hz
48-55	Speed std deviation	`00011000`	24	2.4	Hz
56-64	Direction average	`011101101`	237	237	deg
65-73	Direction of gust	`011110100`	244	244	deg
74-81	Direction std deviation	`00010001`	17	17	deg
82-90	CCW minimum direction	`011000110`	198	198	deg
91-99	CW maximum direction	`100100001`	289	289	deg
100-106	Time of 1s gust	`0100101`	37	185	s
107	Alarm sent	`0`	0	0	
108-112	Debug flags	`01100`	12	12	

Worked in full for the average: bits 10-21 are `001001101001` = 617, and 617 x 0.02 + 0 = 12.34 Hz. That is not yet a wind speed and the two gust fields are not yet gusts - the payload is only half decoded at this point. Sections 3.3 and 3.4 finish it: the 3 s gust is 12.34 + 6.2 = 18.54 Hz and the 1 s gust is 18.54 + 2.8 = 21.34 Hz, which on a current

anemometer come out as 8.43, 12.49 and 14.31 m/s.

3. Four things that break naive decoders

3.1 All-ones means "no reading", not "maximum"

When the firmware has no valid measurement - the sensor failed, the value was NaN, or it fell outside the encodable range - it does not clamp and it does not omit the field. It writes every bit of that field to 1.

A decoder that scales the all-ones pattern reports a plausible-looking extreme instead of a fault: a wind speed average of 81.90 Hz, which converts to about 51 m/s, or a 204.7 Hz alarm gust, about 111 m/s. These are the exact values a broken sensor produces. Test for all-ones BEFORE scaling and emit null.

Field	Bits	All-ones raw	Decodes to	Meaning
Wind speed average	12	4095	81.90 Hz	No valid average
3s gust addition	9	511	51.1 Hz	No gust, or it exceeded the average by over 51.1 Hz
1s gust addition	8	255	25.5 Hz	Over 25.5 Hz above the 3 s gust, or on port 3 below it - see 5.1
3s minimum	9	511	51.1 Hz	No minimum, or the wind never fell below 51.1 Hz
Speed std deviation	8	255	25.5 Hz	Not available
Direction average / gust / CCW / CW	9	511	511 deg	No valid direction
Direction std deviation	8	255	255 deg	Not available
Time of 1s gust	7	127	635 s	Not available
Alarm 3s gust (port 3)	11	2047	204.7 Hz	No valid gust

The rule does not apply to counters, flags and identifiers - the index, the battery and alarm bits, both debug-flags fields, the firmware version and DevAddr - where an all-ones pattern is a legitimate value. The decoder in section 8 uses two different readers, `field()` and `plain()`, for exactly this reason.

One consequence is easy to miss: some fields depend on others. When the average is null the 3 s gust addition goes null with it, but the 1 s gust addition is computed between the two gusts and can still carry a number - which is useless with nothing valid underneath it. When the direction average is null the station transmits 360 for both ends of the direction sector rather than the invalid pattern, so that sector has to be discarded on the strength of the average, not on its own bits.

A null is not always a fault. The average gets 12 bits at 0.02 Hz and reaches 81.90 Hz, but the 3 s minimum gets 9 bits at 0.1 Hz and stops at 51.1 Hz. So in wind that never drops below 51.1 Hz - around 33 m/s - the minimum encodes as all-ones while the average beside it decodes perfectly. A null minimum next to a healthy average means the wind stayed strong, not that the anemometer failed. The gust additions have a ceiling of their own that bites in the opposite conditions - see 3.3.

3.2 The battery bit is not a voltage

The periodic payload carries one bit of battery information. It is 1 only when the battery voltage falls inside the single 200 mV slot selected by `index % 5`:

index % 5	Battery slot the bit reports on
0	3300 - 3500 mV
1	3500 - 3700 mV
2	3700 - 3900 mV
3	3900 - 4100 mV
4	4100 - 4300 mV

So a 1 means "the voltage is inside this payload's slot" and a 0 means "it is not" - which says nothing about which other slot it is in. To recover a voltage a server must watch five consecutive indices and note which one reported 1. Until then the battery state is unknown, and a device that has just joined cannot report a voltage at all for the first few uplinks.

If every one of five consecutive indices reports 0, the voltage is outside 3300-4300 mV entirely - treat that as a fault rather than as missing data.

3.3 The gusts are additions, not speeds

The periodic payload never transmits a gust directly. It transmits the period average, then the 3 s gust as an amount to add to that average, then the 1 s gust as a further amount to add to the 3 s gust. Each field is only meaningful on top of the one before it.

Quantity	Reconstruction
3 s gust	`average + gust_3s_addition`
1 s gust	`average + gust_3s_addition + gust_1s_addition`
3 s minimum	transmitted directly - not an addition

Publishing the raw addition fields as "gust" understates every gust reading, badly. In the worked example of section 2.2 the 3 s gust field reads 6.2 and the real 3 s gust is 18.54 Hz - a factor of three. The error is largest in steady wind, where the additions are small and the average is not, so the readings look calm and internally consistent while being wrong.

The chain is why the fields are small enough to fit: the average gets 12 bits at 0.02 Hz, the additions 9 and 8 bits at 0.1 Hz. It also means the reconstruction accumulates the rounding of every step it passes through:

Quantity	Resolution	Worst-case reconstruction error
Average	0.02 Hz	+/- 0.01 Hz
3 s gust	0.1 Hz on top of the average	+/- 0.06 Hz
1 s gust	0.1 Hz on top of the 3 s gust	+/- 0.11 Hz

The firmware guarantees the 1 s gust is never below the 3 s gust, clamping it twice on the way to the payload, so the 1 s addition on port 1 is never negative. That makes an all-ones there mean the opposite of what it means on port 3: the 1 s gust ran more than 25.5 Hz ABOVE the 3 s gust. Substituting the 3 s gust in that case would understate the peak badly, so leave it null on port 1. On port 3 the clamp is not applied before encoding and the all-ones means a negative difference, where falling back to the 3 s gust is the right move - see 5.1.

The ceiling on the 3 s gust is not a fixed wind speed - it moves with the average. The addition field stops at 51.1 Hz, so the highest gust a payload can carry is the period average plus 51.1 Hz. A squall over an otherwise calm period is the case that breaks it, and it takes the 1 s gust down with it: the 1 s gust is built on the 3 s gust, so when the 3 s addition overflows BOTH gusts decode as null while the average, the direction and the minimum all look perfectly healthy.

Period average	Highest 3 s gust the payload can carry
2 Hz (1.5 m/s)	53.1 Hz (34.2 m/s)
5 Hz (3.6 m/s)	56.1 Hz (36.0 m/s)
10 Hz (6.9 m/s)	61.1 Hz (39.0 m/s)
20 Hz (13.4 m/s)	71.1 Hz (44.9 m/s)
40 Hz (26.2 m/s)	91.1 Hz (56.3 m/s)

Vector averaging makes this more likely, not less. In vector mode the average is the resultant, which falls towards zero as the direction swings, while the gust is still measured on the scalar wind - so the difference the field has to carry grows. A squally, direction-shifting period is exactly where both conditions meet. Nothing can be recovered from the payload when it happens; record it as a gust above the ceiling rather than as no gust at all.

3.4 The payload carries click frequency, not m/s

Every wind speed in every message is transmitted as the anemometer click frequency in Hz, which is the number of cup revolutions per second. Converting it to m/s is the server's job, and the curve to use is a property of the anemometer fitted - it is never transmitted.

Anemometer	Wind speed from click frequency f [Hz]
Current, from 2023	`v = -0.00065*f^2 + 0.675*f + 0.2`
Earlier, before 2023	`v = 0.6335*f + 0.3582`

Applying the wrong curve is a silent error. The two happen to agree around 60 Hz, which makes a mismatch easy to miss on a windy day, and they diverge either side of it - by half a metre per second in the middle of the range and by 2.5 m/s at 100 Hz. Set the curve once, per device, to match the unit installed.

Click frequency	Current anemometer	Earlier anemometer	Difference
10 Hz	6.88 m/s	6.69 m/s	+0.19
20 Hz	13.44 m/s	13.03 m/s	+0.41
40 Hz	26.16 m/s	25.70 m/s	+0.46
60 Hz	38.36 m/s	38.37 m/s	-0.01
80 Hz	50.04 m/s	51.04 m/s	-1.00
100 Hz	61.20 m/s	63.71 m/s	-2.51

The station itself only knows the earlier, linear pair. It uses those two constants internally whenever it converts an alarm threshold between m/s and click frequency, whatever anemometer is fitted. On a current unit that means a threshold you set in m/s takes effect at a slightly different wind speed than the same number decoded from an uplink. Set alarm thresholds in Hz - the downlink of section 7 takes Hz directly - and the discrepancy disappears.

Two details follow whichever curve you use. The firmware reports a still zero below 0.3 Hz rather than the offset term, so a decoder should do the same - otherwise every calm night reads a fifth of a metre per second. And the standard deviation is a spread, not a speed: scale it by the slope of the curve alone, never by the whole curve, which would add the offset to it. For the current anemometer the slope at frequency f is `0.675 - 0.0013*f`.

4. Port 1 - periodic measurement, 14 bytes

The routine uplink, sent every 2 to 10 minutes - 10 by default. Also appears on port 4 as a retransmission. Uses all 112 bits.

Bits	Field	W	Min	Res	Unit	Notes
1-8	Index	8	0	1	-	Revolving 0-255. Identifies the payload for retransmission requests
9	Battery in slot	1	0	1	-	See 3.2 - not a voltage
10-21	Wind speed average	12	0	0.02	Hz	Period average. Scalar or vector - see 4.3
22-30	3s gust addition	9	0	0.1	Hz	ADD to the average to get the 3 s gust - see 3.3
31-38	1s gust addition	8	0	0.1	Hz	ADD to the 3 s gust to get the 1 s gust - see 3.3
39-47	3s minimum	9	0	0.1	Hz	Lowest rolling 3 s average. Absolute, not an addition
48-55	Speed std deviation	8	0	0.1	Hz	Of the 1 s samples. A spread - see 3.4
56-64	Direction average	9	0	1	deg	Period average, 1-360. See 4.2
65-73	Direction of gust	9	0	1	deg	Direction at the moment of the 1 s gust
74-81	Direction std deviation	8	0	1	deg	Of the 1 s direction samples
82-90	CCW minimum direction	9	0	1	deg	Counter-clockwise end of the direction sector - see 4.2
91-99	CW maximum direction	9	0	1	deg	Clockwise end of the direction sector - see 4.2
100-106	Time of 1s gust	7	0	5	s	Seconds from the start of the period, 0-600
107	Alarm sent	1	0	1	-	1 if an alarm payload was sent during the period
108-112	Debug flags	5	0	1	-	Raw 0-31. Not flags in this firmware - see 4.4

4.1 Transmission timing

The station spreads its uplinks to avoid every device in a fleet transmitting at the same instant. It derives a delay of 0 to 29 seconds from the payload itself, so the interval between two uplinks varies by up to half a minute either way around the configured period. Timestamp on receipt; do not assume a fixed cadence, and do not treat a late payload as a missed one until the period plus 29 seconds has passed.

4.2 Direction is 1-360, and the extremes are a sector

Wind direction follows the meteorological convention of 1 to 360 degrees, with 360 meaning north. Zero is never transmitted as a direction: a reading that rounds to zero is reported as 360. A decoder that special-cases 0 as north and 360 as invalid has it exactly backwards.

The CCW minimum and CW maximum are not a numeric minimum and maximum. They are the two ends of the arc the wind swung through during the period, found by walking outwards from the average in each direction - the same pair METAR uses to report a variable wind. The arc runs clockwise from the CCW end to the CW end and always contains the average.

When the sector crosses north the CCW end is numerically LARGER than the CW end - 340 to 20 degrees is a perfectly normal 40-degree sector, not a corrupt payload. Do not sort the pair, and do not compute the width as a plain subtraction: use `(cw - ccw + 360) mod 360`.

4.3 Scalar and vector averaging

The station can average wind either scalar or vector. In vector mode the average speed and the average direction are both taken from the resultant vector, which gives a lower speed than scalar averaging whenever the direction varies. The mode changes nothing about the payload layout, but it does change what two of the fields mean, and the two modes are not comparable in a single time series.
The payload gives no indication of which mode produced it. The mode is reported only in the MW1S service message (section 6.1), so record it per device rather than trying to infer it.

4.4 The debug-flags field does not carry flags

The trailing 5 bits are the field the firmware calls debug flags, and older payload sheets describe them that way. The firmware does not write flags into them. It writes a single scalar, clamped to the field maximum of 31:

Step	What the firmware does
1	Computes `sqrt(mean 1s click count) - mean(sqrt 1s click count)` over the measurement period, and clamps a negative result to zero
2	Divides that by 0.00258, rounds to an integer and clamps it to 31
3	Packs the result into the 5-bit field at resolution 1

So the field value is the raw count 0 to 31. Multiplying it back by 0.00258 recovers the quantity above, which spans 0 to 0.07998 and grows with the gustiness of the flow. Either way it is a number, not a bit field - testing individual bits of it is meaningless.
This use of the field is provisional. The firmware still names it debug flags throughout, still labels it that way in its own diagnostic output, and carries a commented-out assignment of a real flags value beside the line that computes the scalar. Decode it, but do not build anything that depends on the meaning staying as it is.

5. Port 3 - alarm, 6 bytes

Sent when the wind crosses a configured threshold, carrying the readings at the moment of the event rather than period averages. Also appears on port 4 as a retransmission. Uses all 48 bits.

Bits	Field	W	Min	Res	Unit	Notes
1-5	Index	5	0	1	-	Revolving 0-31, a separate counter from the periodic index
6-16	3s gust	11	0	0.1	Hz	ABSOLUTE here, not an addition - see 5.1
17-24	1s gust addition	8	0	0.1	Hz	ADD to the 3 s gust. Can be invalid - see 5.1
25-33	Direction of gust	9	0	1	deg	Direction at the moment of the event, 1-360
34-40	Time of 1s gust	7	0	5	s	Seconds from the start of the period, 0-600
41-48	Alarm flags	8	0	1	-	Bit 0: 1 = high alarm, 0 = low alarm. See 5.2

5.1 The alarm payload is not the periodic payload

The 3 s gust here is the gust itself, at 11 bits and 0.1 Hz, covering 0 to 204.6 Hz. On port 1 the same name is a 9-bit addition to the average. Adding the port 3 value to an average, or reading the port 1 value as a gust, both produce numbers that look reasonable and are wrong.

The 1 s gust addition works the same way on both ports - add it to the 3 s gust - with one difference. On port 1 the firmware floors the difference at zero before encoding; on port 3 it does not, so when the 1 s gust is below the 3 s gust the field encodes as all-ones. Treat that as "1 s gust not available" and fall back to the 3 s gust, rather than discarding the alarm.

The alarm payload carries no average, no minimum, no standard deviation and no battery bit. Everything in it describes a single instant.

5.2 High and low alarms, and what re-arms them

The station runs one alarm at a time, in one of two modes, against the current rolling 3 s wind speed. The two thresholds are not two independent alarms: one fires and the other re-arms.

Mode	Fires when	Re-arms when
High (mode bit 1)	3 s speed rises ABOVE the high threshold	3 s speed falls to or below the LOW threshold
Low (mode bit 0)	3 s speed falls BELOW the low threshold	3 s speed rises to or above the HIGH threshold

So in high mode the low threshold is the hysteresis point, not a second alarm, and a station in high mode will never send a low alarm however calm it gets. Bit 0 of the alarm flags field tells you which mode produced the event; the remaining seven bits are unused and read zero.

After an alarm fires the station is silent for the snooze period, and it will not fire again until the wind has crossed the opposite threshold to re-arm it. A single storm therefore produces one alarm, not a stream - the absence of further alarms is not evidence the wind dropped. Use the periodic payloads for that, and the alarm-sent bit in them to tell that an alarm happened at all.

6. Port 2 - service messages, 12 bytes

Three different messages share port 2 and all three are 12 bytes long. They are sent only in response to a service-message request, never on their own schedule.
MW0S, MW1S and MW2S are all 12 bytes. Read the first 2 bits to tell them apart: 0 is MW0S, 1 is MW1S, 2 is MW2S. Do not branch on length.

6.1 MW1S - device identity

Firmware version, LoRaWAN address, measurement period and averaging mode. Uses 72 of the 96 bits; bits 73-96 are zero padding, not a field.

Bits	Field	W	Notes
1-2	Message type	2	Always 1 for MW1S. This is the discriminator
3-7	Hardware type	5	0 test, 1 MeteoHelix, 2 MeteoWind, 3 MeteoRain, 4 MeteoAG
8-10	FW major	3	Firmware version, first component
11-16	FW minor	6	Second component. Render zero-padded to 2 digits
17-24	FW patch	8	Third component. Render zero-padded to 3 digits
25-56	Device address	32	LoRaWAN DevAddr. Render as 8 hex digits
57-64	Debug	8	Mirrors the averaging mode in this firmware. Do not rely on it
65-71	Measurement period	7	Minutes between periodic uplinks, 2-10
72	Payload type	1	0 scalar averaging, 1 vector averaging - see 4.3
73-96	(padding)	24	Zero fill, not a field

The three version components are separate fields, so `1`, `2`, `11` renders as `1.02.011` - zero-pad the minor to two digits and the patch to three to match how the station names its own version. MW1S is the only message that reports the measurement period and the averaging mode, so request it once per device and record both.

6.2 MW2S - alarm settings readback

The alarm settings currently in effect. It is the only way to confirm that an alarm-settings downlink was accepted, because that command sends no reply of its own. Uses 32 of the 96 bits; bits 33-96 are zero padding.

Bits	Field	W	Min	Res	Unit	Notes
1-2	Message type	2	0	1	-	Always 2 for MW2S
3-11	Alarm snooze	9	0	120	s	Hold-off after an alarm, 120-61320 s
12	Alarm mode	1	0	1	-	1 high alarm, 0 low alarm - see 5.2
13-19	Low threshold	7	0	1	Hz	LOW COMES FIRST here - see 7.2
20-26	High threshold	7	0	1	Hz	0-127 Hz
27-32	Debug	6	0	1	-	Always 0 in this firmware
33-96	(padding)	64	-	-	-	Zero fill, not a field

Both thresholds are click frequencies in Hz and convert to m/s with the curve of section 3.4. Both remain in the payload whatever the mode is, so do not read a threshold as evidence that it is the one that will fire - check the mode bit.

6.3 MW0S - position and inclination, not implemented

MW0S is transmitted, and every value in it is a fixed placeholder compiled into the firmware. No GPS receiver and no inclinometer are fitted to this product. Every station answers with the same coordinates, the same satellite count and the

same inclination, indefinitely.

It is listed here only so that a decoder recognises the message and discards it rather than storing a fleet of stations at one address in Slovakia. The layout below is what the bits contain; do not build features on it. Note also that latitude and longitude are encoded with a minimum of zero, so the format cannot represent southern or western positions at all.

Bits	Field	W	Min	Res	Unit / notes
1-2	Message type	2	0	1	Always 0 for MW0S
3-28	Latitude	26	0	0.00001	deg
29-53	Longitude	25	0	0.00001	deg
54-55	Fix status	2	0	1	bit 0 GNSS, bit 1 DGPS
56-58	Satellites	3	0	1	6 means 6 or more
59-66	HDOP	8	0	0.1	-
67-74	Inclination X	8	0	0.25	deg
75-82	Inclination Y	8	0	0.25	deg
83-90	Inclination Z	8	0	0.25	deg
91-96	Debug	6	0	1	-

7. Port 10 - alarm-settings downlink, 3 bytes

This is the one downlink built the same way as the uplinks: 24 bits packed most-significant-bit first, filling 3 bytes exactly. It replaces all four alarm settings at once.

Property	Value
Port	10
Payload	3 bytes
Reply	None - see 7.4
Effect	Replaces the snooze, the mode and both thresholds at once. The station keeps them through a power cycle
Device reset	No

The station is a Class A LoRaWAN device: it only listens briefly after it transmits. A downlink queued on the network server is delivered after the station's next uplink, which can be up to the configured measurement period away - by default 10 minutes, plus up to 29 seconds of transmit spreading. Queue one command at a time and wait for it to be delivered.

Bits	Field	W	Min	Res	Valid range the firmware enforces
1-9	Alarm snooze	9	0	120	120 - 61320 s, so raw 1-511. Raw 0 is rejected
10	Alarm mode	1	0	1	1 high alarm, 0 low alarm
11-17	High threshold	7	0	1	0 - 127 Hz. HIGH COMES FIRST here - see 7.2
18-24	Low threshold	7	0	1	0 - 127 Hz, and never above the high threshold

7.1 Encoding

Encoding is the parsing model of section 2 run backwards: for each field, `raw = round((value - minimum) / resolution)`, then append that raw value as the stated number of bits, most significant bit first. The thresholds are click frequencies in Hz, so convert from m/s with the curve of section 3.4 before encoding, not after.

The all-ones sentinel of section 3.1 does NOT apply to this payload. It is an uplink convention for reporting a failed sensor. Here an all-ones field is simply a large number, and a large number out of range gets the command rejected. Never write all-ones as a "not set" marker.

The 7-bit threshold fields cap both thresholds at 127 Hz. The firmware's own range check allows up to 160 Hz, but no value above 127 can be encoded, so 127 is the real ceiling - about 75 m/s on a current anemometer.

7.2 The field order is not the same as MW2S

MW2S reports the LOW threshold first and the HIGH threshold second. This downlink carries the HIGH threshold first and the LOW threshold second. The two fields are the same width and sit next to each other, so swapping them produces a valid command that silently sets the wrong pair - and in high mode a low threshold that is really the high one will re-arm the alarm immediately, so the station simply stops alarming.

This is the single most common mistake against this protocol. Unlike the other products in the range, a MeteoWind settings readback cannot be truncated and sent straight back as a command - the fields have to be reordered.

7.3 Validation is all-or-nothing and silent

If any single field falls outside the range in the table, the firmware discards the ENTIRE command and keeps the previous settings. There is no reply either way, so a rejected write looks exactly like a downlink that never arrived. Range-check before sending and confirm afterwards by requesting MW2S.

The pairwise rule catches people out: a low threshold above the high threshold is rejected even when both are individually in range. Check that relationship before sending, not just the two bounds.

7.4 Confirming a write

Because the command sends no reply, the only confirmation is to request the MW2S service message and compare the settings it reports against what you sent - remembering that MW2S lists the thresholds in the opposite order.

7.5 Worked example

Snooze 600 s; high-alarm mode; high threshold 47 Hz, about 30 m/s on a current anemometer; low threshold 10 Hz - about 6.9 m/s - the point at which the alarm re-arms:

Encoding	Value
Downlink (3 B, port 10)	`02D78A`
Base64 for the network server	`AteK`

Field by field: snooze 600/120 = 5 in 9 bits; mode 1 in 1 bit; high 47 in 7 bits; low 10 in 7 bits. That is `000000101 1 0101111 0001010`, which regrouped into bytes is `00000010 11010111 10001010` = `02D78A`. The same settings read back as MW2S are `80B14BC000000000000000000` - the same numbers in a different order, with the message type in front.

8. Reference decoder

The decoder below is supplied alongside this document as the file `meteowind_gen2_decoder.js`. Use that file rather than retyping from this page.
It is written in ES5 with no dependencies, so it pastes directly into a The Things Stack v3 or ChirpStack v4 payload-formatter box; both call `decodeUplink({bytes, fPort})`. Set `ANEMOMETER` at the top to match the unit before using it; it defaults to the current anemometer. The decoder returns both the transmitted frequencies and the reconstructed gusts and speeds, so the intermediate values stay available for checking.
This decoder was verified field by field against firmware 1.02.011 using the test vectors in section 8.1, including the sensor-failure case. Run them through your own decoder before trusting it.

8.1 Test vectors

Feed these through any decoder before trusting it. They are generated from an encoder that mirrors the firmware's bit packer. Speeds in m/s use the current anemometer curve.

Port	Payload (hex)	Expected
1	`039348F8709030ED7A08B1A4294C`	index 3, average 12.34 Hz (8.43 m/s), 3 s gust 18.54 Hz (12.49 m/s), 1 s gust 21.34 Hz (14.31 m/s), 3 s minimum 7.2 Hz, std dev 2.4 Hz, direction 237 deg, gust from 244 deg, sector 198-289 deg, gust at 185 s, no alarm, debug flags 12
1	`077FFFFFFFFFFFFFFFFFFFFFC0`	index 7, every wind speed and direction field null (sensor failure), battery bit 0, no alarm, debug flags 0
1	`00000000000000168B4005A2D0000`	index 0, dead calm: all speeds 0 Hz and 0 m/s, direction 360 deg (north) with a zero-width sector, debug flags 0
3	`2A0F3E9B9301`	index 5, HIGH alarm, 3 s gust 52.7 Hz (33.97 m/s), 1 s gust 58.9 Hz (37.70 m/s), direction 311 deg, 95 s into the period
4	`039348F8709030ED7A08B1A4294C`	Same as the first row, flagged as a retransmission
2	`44420B2700BE440115000000`	MW1S, MeteoWind, firmware 1.02.011, DevAddr 2700BE44, 10 minute period, vector averaging
2	`80B14BC00000000000000000`	MW2S, snooze 600 s, high-alarm mode, low threshold 10 Hz, high threshold 47 Hz
2	`049780C0D0D5975EC8028080`	MW0S, the fixed placeholder values of section 6.3. Recognise and discard
10 dwn	`02D78A`	Alarm-settings downlink, section 7. Same settings as the MW2S row above, in the opposite order. Pass to `decodeAlarmDownlink()`, not `decodeUplink()`

8.2 Source

```
// ============================================================================
//  MeteoWind IoT Pro Gen2 - LoRaWAN uplink payload decoder
//  Barani Design Technologies
//
//  For firmware 1.02.011. Field layouts, names, resolutions and ranges are
//  taken from the firmware. See the Payload Decoding Guide for the details.
//
//  Works as-is as a TTN v3 / ChirpStack v4 JavaScript codec: paste the whole
//  file into the payload-formatter box. decodeUplink({bytes, fPort}) is the
//  entry point both stacks call.
//
//  Ports: 1 periodic (14 B), 2 service MW0S/MW1S/MW2S (12 B), 3 alarm (6 B),
//         4 retransmission (14 or 6 B), 50 command reply (raw hex).
//
//  decodeAlarmDownlink(bytes) is also exported. It parses the 3-byte port 10
//  alarm-settings DOWNLINK, for checking a command before sending it or
//  reading one back from a log. TTN and ChirpStack never call it themselves.
// ============================================================================

// ANEMOMETER: which calibration curve to apply when converting the transmitted
// click frequency f [Hz] to m/s. The payload carries frequency only - the curve
```

```
// is a property of the anemometer fitted, not of the message.
//
//   'post2023' (default)   v = -0.00065*f^2 + 0.675*f + 0.2
//   'pre2023'              v =  0.6335*f + 0.3582
//
// Set this to match the unit. Using the wrong one is a silent error: the curves
// agree near 60 Hz and diverge either side of it, by 2.5 m/s at 100 Hz.
//
// Note the firmware itself always uses the 'pre2023' constants internally
// (WIND_SPEED_CALIB_P0 / P1) when it converts alarm thresholds between m/s and
// click frequency, whichever anemometer is fitted. So on a post-2023 unit a
// threshold set in m/s lands at a slightly different wind speed than the same
// number decoded from an uplink. Set alarm thresholds in Hz to avoid this.
var ANEMOMETER = 'post2023';
// The firmware reports a still zero below this click rate rather than the
// curve's offset term (MINIMUM_CLICKS__1_SEC).
var CALM_HZ = 0.3;

// ------------------------------------------------------------------ bit reader
function Reader(bytes) {
  this.b = bytes;
  this.pos = 0;
}
Reader.prototype.bits = function (n) {
  var v = 0;
  for (var i = 0; i < n; i++) {
    var byteNum = (this.pos + i) >> 3;
    var bitNum = 7 - ((this.pos + i) & 7);
    v = v * 2 + ((this.b[byteNum] >> bitNum) & 1);
  }
  this.pos += n;
  return v;
};

// Read a field and scale it. Returns null when every bit is set, which the
// firmware uses to mean "no valid reading" - not a maximum value.
function field(r, bits, min, res) {
  var raw = r.bits(bits);
  if (raw === Math.pow(2, bits) - 1) return null;
  return round4(raw * res + min);
}

// Same, but the all-ones pattern is a legitimate value (counters, flags, IDs).
function plain(r, bits, min, res) {
  var raw = r.bits(bits);
  if (min === undefined) return raw;
  return round4(raw * res + min);
}

function round4(x) {
  return Math.round(x * 10000) / 10000;
}

function round2(x) {
  return Math.round(x * 100) / 100;
}

// Click frequency [Hz] -> wind speed [m/s].
function hzToMs(hz) {
  if (hz === null || hz === undefined) return null;
  if (hz <= CALM_HZ) return 0;
  if (ANEMOMETER === 'pre2023') return round2(0.6335 * hz + 0.3582);
  return round2(-0.00065 * hz * hz + 0.675 * hz + 0.2);
}

// Slope of the calibration curve at a given frequency. A standard deviation is
// a spread, not a speed, so it scales by the slope alone - never by the whole
// curve, which would add the offset term to it.
function hzSlope(hz) {
  if (ANEMOMETER === 'pre2023') return 0.6335;
  return 0.675 - 0.0013 * hz;
}

// Add two fields, propagating null.
function sum(a, b) {
  if (a === null || b === null) return null;
  return round4(a + b);
}

// ------------------------------------------------------------- port 1 and 4
function decodePeriodic(r) {
  var o = {};
  o.index = plain(r, 8);
  o.batteryInSlot = plain(r, 1);

  o.windSpeedAvg_Hz = field(r, 12, 0, 0.02);
  // The two gusts are transmitted as differences, each on top of the previous
```

```
  // value. Neither is usable on its own - see the guide.
  o.gust3sAddition_Hz = field(r, 9, 0, 0.1);
  o.gust1sAddition_Hz = field(r, 8, 0, 0.1);
  o.windSpeedMin3s_Hz = field(r, 9, 0, 0.1);
  o.windSpeedStdev1s_Hz = field(r, 8, 0, 0.1);

  o.windDirAvg_deg = field(r, 9, 0, 1);
  o.windDirGust_deg = field(r, 9, 0, 1);
  o.windDirStdev_deg = field(r, 8, 0, 1);
  o.windDirCcwMin_deg = field(r, 9, 0, 1);
  o.windDirCwMax_deg = field(r, 9, 0, 1);
  o.gustTime_s = field(r, 7, 0, 5);
  o.alarmSent = plain(r, 1);
  // Field is MW2024_DEBUG_FLAGS in the firmware: 5 bits, resolution 1, so the
  // raw count 0-31 is the field value. The firmware does not write flags into
  // it - it writes roundf(wind_sqrtf_delta / 0.00258f), clamped to 31, where
  // wind_sqrtf_delta = sqrt(mean 1 s click count) - mean(sqrt 1 s click count).
  // Multiply back by 0.00258 to recover that quantity.
  o.debugFlags = plain(r, 5);
  o.sqrtDelta = round4(o.debugFlags * 0.00258);

  // Absolute gusts, rebuilt from the chain of additions.
  o.windSpeedGust3s_Hz = sum(o.windSpeedAvg_Hz, o.gust3sAddition_Hz);
  // No fallback to the 3 s gust here, unlike port 3. The firmware floors this
  // difference at zero before encoding, so an all-ones means the 1 s gust was
  // more than 25.5 Hz ABOVE the 3 s gust (or was itself flagged invalid).
  // Substituting the 3 s gust would understate it badly; null is honest.
  o.windSpeedGust1s_Hz = sum(o.windSpeedGust3s_Hz, o.gust1sAddition_Hz);

  // The same speeds in m/s, which is what most backends want to store.
  o.windSpeedAvg_ms = hzToMs(o.windSpeedAvg_Hz);
  o.windSpeedGust3s_ms = hzToMs(o.windSpeedGust3s_Hz);
  o.windSpeedGust1s_ms = hzToMs(o.windSpeedGust1s_Hz);
  o.windSpeedMin3s_ms = hzToMs(o.windSpeedMin3s_Hz);
  // Scaled by the local slope of the curve, taken at the period average.
  o.windSpeedStdev1s_ms =
    o.windSpeedStdev1s_Hz === null || o.windSpeedAvg_Hz === null
      ? null
      : round2(hzSlope(o.windSpeedAvg_Hz) * o.windSpeedStdev1s_Hz);

  // One payload cannot give a voltage - see the battery section of the guide.
  o.batteryHint_mV = o.batteryInSlot
    ? { min: 3300 + 200 * (o.index % 5), max: 3300 + 200 * (o.index % 5) + 200 }
    : null;
  return o;
}
// ------------------------------------------------------------ port 3 and 4
function decodeAlarm(r) {
  var o = {};
  o.index = plain(r, 5);
  // Unlike the periodic payload, the 3 s gust here is absolute.
  o.windSpeedGust3s_Hz = field(r, 11, 0, 0.1);
  o.gust1sAddition_Hz = field(r, 8, 0, 0.1);
  o.windDirGust_deg = field(r, 9, 0, 1);
  o.gustTime_s = field(r, 7, 0, 5);
  var flags = plain(r, 8);
  o.flags = flags;
  o.alarmType = (flags & 1) === 1 ? 'high' : 'low';

  // Port 3 does not floor this difference before encoding, so a 1 s gust that
  // came in below the 3 s gust arrives as all-ones. A 1 s peak is never really
  // below the 3 s mean it sits in, and the firmware enforces that on port 1, so
  // fall back to the 3 s gust rather than dropping the reading entirely.
  if (o.gust1sAddition_Hz === null && o.windSpeedGust3s_Hz !== null) {
    o.windSpeedGust1s_Hz = o.windSpeedGust3s_Hz;
    o.gust1sFellBackTo3s = true;
  } else {
    o.windSpeedGust1s_Hz = sum(o.windSpeedGust3s_Hz, o.gust1sAddition_Hz);
  }
  o.windSpeedGust3s_ms = hzToMs(o.windSpeedGust3s_Hz);
  o.windSpeedGust1s_ms = hzToMs(o.windSpeedGust1s_Hz);
  return o;
}

// ------------------------------------------------------------------ port 2
var DEVICE_TYPE = ['test', 'MeteoHelix', 'MeteoWind', 'MeteoRain', 'MeteoAG'];

// MW0S carries fixed placeholder values in this firmware - no GPS or
// inclinometer is fitted. It is decoded for completeness only.
function decodeMw0s(r) {
  var o = { messageType: 'MW0S' };
  o.latitude_deg = field(r, 26, 0, 0.00001);
  o.longitude_deg = field(r, 25, 0, 0.00001);
  var fix = plain(r, 2);
  o.fixStatus = { value: fix, gnss: (fix & 1) === 1, dgps: (fix & 2) === 2 };
  o.satellites = plain(r, 3); // 6 means "6 or more"
  o.hdop = field(r, 8, 0, 0.1);
```

```
  o.inclinationX_deg = field(r, 8, 0, 0.25);
  o.inclinationY_deg = field(r, 8, 0, 0.25);
  o.inclinationZ_deg = field(r, 8, 0, 0.25);
  o.debugFlags = plain(r, 6);
  o.placeholder = true;
  return o;
}

function decodeMw1s(r) {
  var o = { messageType: 'MW1S' };
  var hw = plain(r, 5);
  o.hardwareTypeValue = hw;
  o.hardwareType = DEVICE_TYPE[hw] !== undefined ? DEVICE_TYPE[hw] : 'unknown';
  var maj = plain(r, 3), min = plain(r, 6), pat = plain(r, 8);
  o.firmware = maj + '.' + pad2(min) + '.' + pad3(pat);
  o.devAddr = hex8(plain(r, 32));
  o.debugFlags = plain(r, 8);
  o.measurementPeriod_min = plain(r, 7);
  var pt = plain(r, 1);
  o.payloadTypeValue = pt;
  o.payloadType = pt === 1 ? 'vector' : 'scalar';
  // Remaining 24 bits are zero padding - do not read them.
  return o;
}

function decodeMw2s(r) {
  var o = { messageType: 'MW2S' };
  o.alarmSnooze_s = plain(r, 9, 0, 120);
  var mode = plain(r, 1);
  o.alarmModeValue = mode;
  o.alarmMode = mode === 1 ? 'high' : 'low';
  // NOTE the order: LOWER comes first here, UPPER first in the downlink.
  o.thresholdLow_Hz = plain(r, 7);
  o.thresholdHigh_Hz = plain(r, 7);
  o.debugFlags = plain(r, 6);
  o.thresholdLow_ms = hzToMs(o.thresholdLow_Hz);
  o.thresholdHigh_ms = hzToMs(o.thresholdHigh_Hz);
  // Remaining 64 bits are zero padding - do not read them.
  return o;
}
// ------------------------------------------------------- port 10 downlink
// The alarm-settings DOWNLINK (3 bytes). Not an uplink - this is here so a
// backend can check what it is about to send, or read back a logged command.
//
// The all-ones sentinel of the uplinks does NOT apply here. This is a
// command: out-of-range values are rejected by the firmware, not flagged.
function decodeAlarmDownlink(bytes) {
  if (bytes.length !== 3) {
    return { data: {}, errors: ['alarm downlink needs 3 bytes, got ' + bytes.length] };
  }
  var r = new Reader(bytes), o = {};
  o.alarmSnooze_s = plain(r, 9, 0, 120);
  var mode = plain(r, 1);
  o.alarmModeValue = mode;
  o.alarmMode = mode === 1 ? 'high' : 'low';
  // UPPER before LOWER - the opposite of the MW2S readback.
  o.thresholdHigh_Hz = plain(r, 7);
  o.thresholdLow_Hz = plain(r, 7);
  o.thresholdHigh_ms = hzToMs(o.thresholdHigh_Hz);
  o.thresholdLow_ms = hzToMs(o.thresholdLow_Hz);
  return { data: o, warnings: rangeCheck(o) };
}

// The firmware validates all-or-nothing: one field out of range and the whole
// command is discarded, silently. Check before sending.
function rangeCheck(o) {
  var w = [];
  if (o.alarmSnooze_s < 120 || o.alarmSnooze_s > 61320)
    w.push('snooze ' + o.alarmSnooze_s + ' s outside 120-61320 (raw 1-511)');
  if (o.thresholdLow_Hz > 127)
    w.push('low threshold above the 127 Hz the 7-bit field can carry');
  if (o.thresholdHigh_Hz > 127)
    w.push('high threshold above the 127 Hz the 7-bit field can carry');
  if (o.thresholdLow_Hz > o.thresholdHigh_Hz)
    w.push('low threshold above high threshold');
  if (w.length) w.push('THE WHOLE COMMAND WOULD BE DISCARDED by the device');
  return w;
}

// Build a port 10 command. Returns a 3-byte array.
function encodeAlarmDownlink(snooze_s, mode, high_Hz, low_Hz) {
  var bits = [];
  function put(raw, n) {
    for (var i = n - 1; i >= 0; i--) bits.push((raw >> i) & 1);
  }
  put(Math.round(snooze_s / 120), 9);
  put(mode === 'high' || mode === 1 ? 1 : 0, 1);
```

```
  put(Math.round(high_Hz), 7);
  put(Math.round(low_Hz), 7);
  var out = [];
  for (var i = 0; i < 24; i += 8) {
    var b = 0;
    for (var j = 0; j < 8; j++) b = (b << 1) | bits[i + j];
    out.push(b);
  }
  return out;
}

function pad2(n) { return (n < 10 ? '0' : '') + n; }
function pad3(n) { return (n < 10 ? '00' : n < 100 ? '0' : '') + n; }
function hex8(n) {
  var s = (n >>> 0).toString(16).toUpperCase();
  while (s.length < 8) s = '0' + s;
  return s;
}

// ---------------------------------------------------------------- dispatch
function decodeUplink(input) {
  var b = input.bytes, port = input.fPort;
  var r = new Reader(b);
  var data, warnings = [];
  if (port === 1) {
    if (b.length !== 14) return err('port 1 needs 14 bytes, got ' + b.length);
    data = decodePeriodic(r);
  } else if (port === 3) {
    if (b.length !== 6) return err('port 3 needs 6 bytes, got ' + b.length);
    data = decodeAlarm(r);
  } else if (port === 4) {
    // A replay of a port 1 or port 3 payload. Length is the only discriminator.
    if (b.length === 14) { data = decodePeriodic(r); data.retransmitted = true; }
    else if (b.length === 6) { data = decodeAlarm(r); data.retransmitted = true; }
    else return err('port 4 needs 14 or 6 bytes, got ' + b.length);
  } else if (port === 2) {
    // MW0S, MW1S and MW2S are all 12 bytes - the leading 2 bits tell them apart.
    if (b.length !== 12) return err('port 2 needs 12 bytes, got ' + b.length);
    var type = r.bits(2);
    if (type === 0) {
      data = decodeMw0s(r);
      warnings.push('MW0S carries fixed placeholder values - do not store them');
    } else if (type === 1) data = decodeMw1s(r);
    else if (type === 2) data = decodeMw2s(r);
    else return err('unknown service message type ' + type);
  } else if (port === 50) {
    // Configuration command replies. Correlate with the command just sent -
    // these carry no command ID.
    return { data: { commandReply: bytesToHex(b) },
             warnings: ['port 50 reply - identify it from the command you sent'] };
  } else {
    return err('unhandled port ' + port);
  }
  return { data: data, warnings: warnings };
}

function err(m) { return { data: {}, errors: [m] }; }

function bytesToHex(b) {
  var s = '';
  for (var i = 0; i < b.length; i++) {
    var h = b[i].toString(16).toUpperCase();
    s += (h.length < 2 ? '0' : '') + h;
  }
  return s;
}

// Node/CommonJS export - harmless in TTN and ChirpStack, which ignore it.
if (typeof module !== 'undefined') {
  module.exports = {
    decodeUplink: decodeUplink,
    decodeAlarmDownlink: decodeAlarmDownlink,
    encodeAlarmDownlink: encodeAlarmDownlink
  };
}
```

9. Migrating an existing decoder

If you already decode an earlier MeteoWind, or built a decoder from an older payload sheet, these are the points where that decoder will be wrong against firmware 1.02.011. The field positions and widths themselves are unchanged.

9.1 Invalid readings are probably being published as real ones

Older payload sheets scale every field unconditionally, so a decoder written from one converts the all-ones pattern into a physical value instead of recognising it as a fault. A station with a failed anemometer then reports a steady 51 m/s, and a failed direction sensor reports 511 degrees, indefinitely.
This is the most consequential difference and the easiest to miss, because nothing looks broken - the payloads arrive on schedule and the numbers are in range for the field, if not for the weather. Implement the all-ones check of section 3.1 before anything else.

9.2 Gusts published from the raw fields are far too low

A field table alone does not say that the gust fields are additions, and a decoder built from one publishes them directly. The result understates every gust, by the whole period average - which is most of the value. Apply the reconstruction of section 3.3, and check an existing archive for gusts that are implausibly close to zero while the average is not.

9.3 Smaller points

Item	Note
Anemometer curve	A decoder built on the linear curve under-reads a current anemometer below 60 Hz and over-reads it above, by up to 2.5 m/s at the top of the range. Check which curve an existing integration uses. See 3.4
Trailing 5 bits of port 1	Still named debug flags, and still 5 bits at resolution 1, but this firmware writes a gustiness scalar into them rather than flags. A decoder that unpacks them bit by bit reports nonsense. See 4.4
MW1S bit 72	Reserved for future use in some earlier sheets. It now carries the averaging mode, 0 scalar and 1 vector, and is the only place that mode is reported
Alarm payload flags	Eight bits, of which only bit 0 is used: 1 high alarm, 0 low alarm. Earlier sheets describe the field as unused
Port 10 field order	Unchanged, and still the opposite of MW2S. If an existing integration works, do not "fix" it to match the readback order. See 7.2

10. Decoder checklist

#	Check
1	Read bit ranges, not bytes
2	Test every measurement field for all-ones and emit null - do not scale it
3	Do not apply the all-ones rule to counters, flags, the debug-flags fields, the firmware version or DevAddr
4	Rebuild the 3 s gust as average + addition, and the 1 s gust as 3 s gust + addition - never publish the addition fields as gusts
5	Discard the gusts when the average is null, and the direction sector when the direction average is null
6	Do not raise a sensor fault on a null 3 s minimum beside a valid average - the minimum only reaches 51.1 Hz and strong wind overruns it

#	Check
7	Expect BOTH gusts to go null together when a squall exceeds the average by more than 51.1 Hz, and record that as a gust over the ceiling, not as no gust
8	Convert Hz to m/s with the calibration for the anemometer fitted, and set speeds at or below 0.3 Hz to calm
9	Scale the speed standard deviation by the slope of the curve, not the curve
10	Treat direction 360 as north and 0 as never transmitted
11	Allow the CCW end of the direction sector to exceed the CW end, and take the width modulo 360
12	Use the port 3 encoding for port 3 - the 3 s gust there is absolute and 11 bits wide
13	Accept an all-ones 1 s gust addition on port 3 and fall back to the 3 s gust
14	Branch port 2 on the leading 2 bits, not on length - all three service messages are 12 bytes
15	Recognise MW0S and discard it - its contents are fixed placeholders
16	Decode port 4 by length: 14 bytes periodic, 6 bytes alarm
17	Collect five consecutive indices before reporting a battery voltage
18	Expect one alarm per storm, not a stream - the snooze and the re-arm threshold suppress the rest
19	When ENCODING the port 10 downlink, put the HIGH threshold first, keep low below high, range-check every field - one bad field silently discards the whole command - and never use all-ones as a "not set" marker there